

Session: Artificial Neural Networks

Javier Serrano

Applied Machine Learning
Master in Data Science and Analytics



Strathmore University

@iLabAfrica Centre

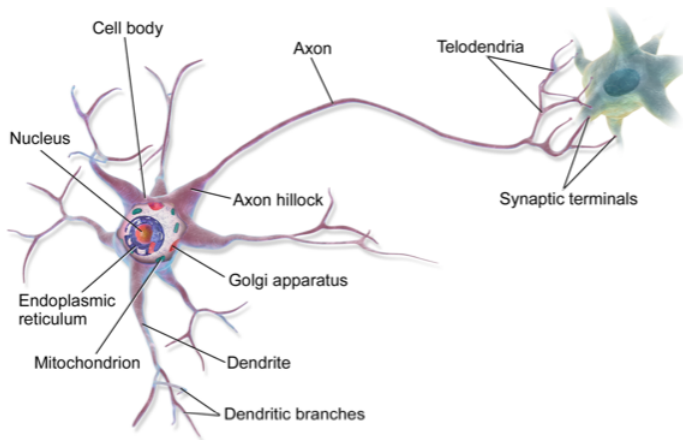
Summary

The Perceptron

Fully Connected Neural Networks

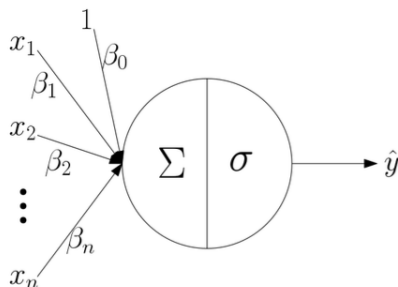
Training a Dense Neural Network

A Biological Neuron



The Logistic Regression

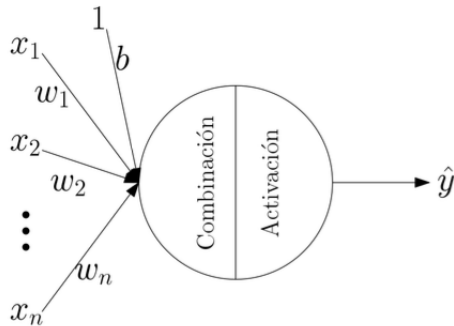
$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)}}$$



- Σ represents the weighted sum of the inputs the parameters β
- σ represents the sigmoid function

The Perceptron

The Simplest ANN Architecture



- In the original Perceptron the activation function was a STEP

Activation Functions

Must be non-linear

Sigmoid Function $y(x) = \frac{1}{1 + e^{-x}}$, The outputs are always in $(0, 1)$

Hyperbolic Tangent Function $y(x) = \tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$, The outputs are in $(-1, 1)$.
Used in Recurrent NN

ReLu Function $y(x) = \max(0, x)$, Very easy to compute and get good results.

Summary

The Perceptron

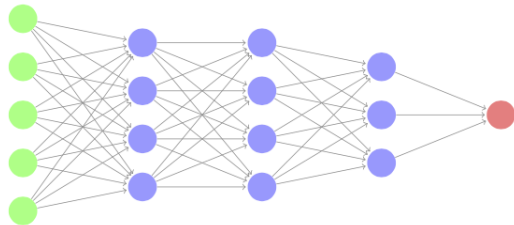
Fully Connected Neural Networks

Training a Dense Neural Network

Fully Connected Neural Networks

Fully Connected = Dense

- One input layer representing the input data x
- One output layer with one neuron producing the aoutput of the network: a number or a class
- Three hidden layers computing the output
- The **architecture of the network** is established setting the number of layers and neurons per layer

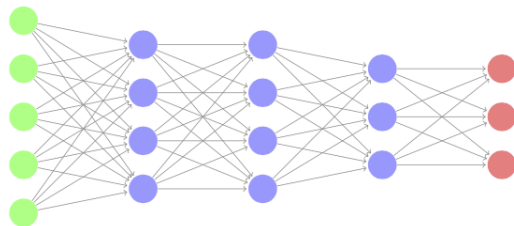


Fully Connected Neural Networks

Classification Problem

- The number of neurons in the input layer is determined by the number of variables/attributes in the dataset $\mathcal{X}$
- The number of neurons in the output layer is **1 in regression problems** and is determined by the **number of categories in classification problems**
- Activation function in the output neurons: **Softmax**

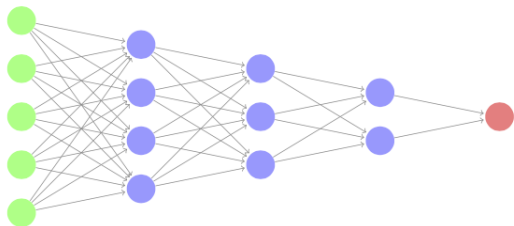
$$S(z)_j = \frac{e^{z_j}}{\sum_{k=1}^c e^{z_k}}$$



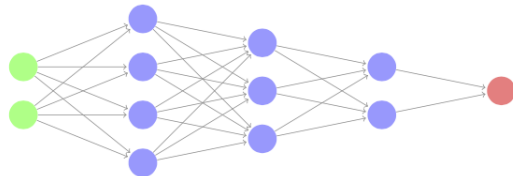
Fully Connected Neural Networks

Architecture with high/low dimensionality (atributes) in the inputs

High Dimensionality

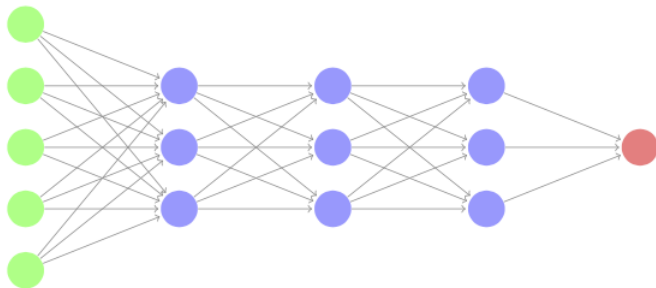


Low Dimensionality



Fully Connected Neural Networks

Nowadays, this is very common architecture



The hidden layer transform the input data in abstract concepts.

Summary

The Perceptron

Fully Connected Neural Networks

Training a Dense Neural Network

Training a Dense Neural Network

Let's consider a NN with L layers, n_i neurons in layer i and activation function g .

- Forward Pass
- Parameter initialization
- Gradient Descent
- Backward propagation

Training a Dense Neural Network

Forward propagation

For each layer l , we can write the parameters that connect the neurons from layer l to layer $l - 1$ in a matrix $W^{[l]} \in M_{n_l \times n_{l-1}}(\mathbb{R})$, where n_l and n_{l-1} denote the number of neurons in layers l and $l - 1$

The process of obtaining the output of a neural network given an input $x \in \mathcal{X}$ from the dataset is known as **forward propagation**

- Input: $x \in \mathcal{X}$
 1. We denote $a^{[0]} = x$
 2. For l from 1 to L we calculate:
 - 2.1 $z^{[l]} = W^{[l]} * a^{[l-1]} + b^{[l]}$
 - 2.2 $a^{[l]} = g(z^{[l]})$
 3. We denote $\hat{y} = a^{[L]}$
- Output: $\hat{y}$

Training a Dense Neural Network

Parameter Initialization

- **Break the Symmetry:** random initialization of every neuron weight
- If we initialize all the neurons with the same value, all the parameters would always be modified in the same way, which would eventually lead to all the neurons having the same weights. This would result in a neural network that is equivalent to having a single neuron in each layer
- A good way of initialize the weights are:
 1. For a neuron with a sigmoid activation function
 - Uniform distribution $U(-r, r)$, with $r = \sqrt{\frac{6}{n_{in} + n_{out}}}$
 - Normal distribution $N(0, \sigma^2)$, with $\sigma^2 = \frac{2}{n_{in} + n_{out}}$
 2. For a neuron with a ReLu activation function
 - Uniform distribution $U(-r, r)$, with $r = \sqrt{\frac{6}{n_{in}}}$
 - Normal distribution $N(0, \sigma^2)$, with $\sigma^2 = \frac{2}{n_{in}}$

Training a Dense Neural Network

Fitting the parameters: Gradient Descent

- To measure the performance of the neural network in modeling the data $(\mathcal{X}, \mathcal{Y})$, we define a **cost function**

1. For regression problems, the mean squared error (MSE)

$$\mathcal{L} = (y - \hat{y})^2$$

2. For classification problems, the Cross Entropy

$$\mathcal{L} = \sum_{y=1}^c y_i \log p_i$$

- And the cost function to the whole set of data

$$J = \frac{1}{m} \sum_{x \in \mathcal{X}} \mathcal{L}(x), m \text{ number of instances in dataset}$$

Training a Dense Neural Network

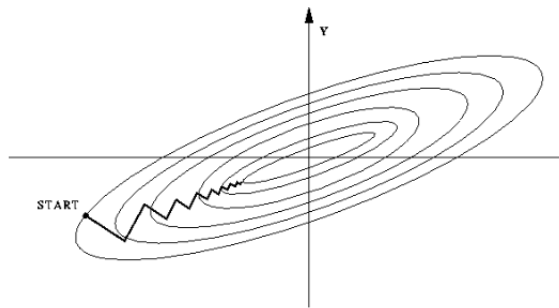
Gradient Descent as usual

- We compute the gradient of the cost function

$$\nabla J = \left(\frac{\partial J}{\partial w_{11}^{[1]}}, \frac{\partial J}{\partial w_{12}^{[1]}}, \dots, \frac{\partial J}{\partial w_{ij}^{[l]}}, \dots, \frac{\partial J}{\partial b_i^{[l]}}, \dots \right)$$

- We update the parameters in the opposite direction of gradients

$$\theta = \theta - \eta \nabla_{\theta} J$$



Training a Dense Neural Network

How to compute the Gradients: Backpropagation

- The backpropagation algorithm is based on the **chain rule**.
- Backpropagation Algorithm
 - **Input:** $y \in \mathcal{Y}$ and $a^{[L]}$
 - Calculate $\mathcal{L}(y, a^{[L]})$ and $\frac{\partial \mathcal{L}}{\partial a^{[L]}}$
 - For l from L to 1:
 - a. Calculate $\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \left(\frac{\partial \mathcal{L}}{\partial a^{[l]}} \cdot g'(z^{[l]}) \right) * (a^{[l-1]})^T$
 - b. Calculate $\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \left(\frac{\partial \mathcal{L}}{\partial a^{[l]}} \cdot g'(z^{[l]}) \right)$
 - c. Calculate $\frac{\partial \mathcal{L}}{\partial a^{[l]}} = (W^{[l]})^T * \left(\frac{\partial \mathcal{L}}{\partial a^{[l+1]}} \cdot g'(z^{[l+1]}) \right)$
 - **Output:** $\frac{\partial \mathcal{L}}{\partial W^{[l]}}$ and $\frac{\partial \mathcal{L}}{\partial b^{[l]}}$ for all $l = 1, \dots, L$

References

- A. Geron, *Hands-on Machine Learning with Scikit-Learn, Keras, and Tensor-Flow*, O'Reilly, 2019.
- Andrew Ng course on Deep Learning
<https://www.youtube.com/watch?v=E13qqHb3J7U>
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
<https://github.com/janishar/mit-deep-learning-book-pdf>
- Simon J.D. Prince *Understanding Deep Learning* 2024
<https://udlbook.github.io/udlbook/>